

Enterprise LMS

System Design · Design Patterns · Stack — Brief

Prepared by Gs Dwivedi · 21 June 2026

1. System Design

A Next.js front end calls an API Gateway (Spring Cloud Gateway), which routes to independent Spring Boot microservices that communicate asynchronously over Kafka, each owning its own database.

Services — IAM (authentication and dynamic RBAC), Org & Tenant, Course, Batch & Enrolment, Assessment, Engagement, Approval, Notification, and Media / Streaming.

Data & infrastructure — PostgreSQL per service, Redis (cache, sessions, permission cache), object storage and CDN (video and documents), and Elasticsearch (search); plus service discovery, a config server, and a secrets vault.

Core ideas — The JWT carries identity and roles only; each service resolves and caches permissions; access is data-driven and reconfigurable at runtime; resilience patterns wrap every inter-service call; cross-service consistency uses events and sagas.

2. Design Patterns

Object / class (inside a service) — Dependency Injection, Repository, Strategy (channel adapters, assessment types), Factory, Builder (queries, JasperReports documents), Decorator (caching / retry), Adapter (Jasper / WhatsApp / email SDKs), Facade, and Observer (Spring events).

Architecture (per service) — Layered with modular boundaries, Hexagonal / Clean architecture, CQRS (course authoring vs delivery reads), DDD tactical patterns (aggregates, value objects), and database-per-service.

Distributed (between services) — API Gateway and BFF, Saga with compensation (e.g. trainer provisioning across IAM + Org), Transactional Outbox, Circuit Breaker, Retry with back-off, Timeout, Bulkhead, Service Discovery, externalised config, and Anti-Corruption Layer.

Data & messaging (cross-cutting) — Cache-aside (permissions, catalogue), read replicas, Pub/Sub event-driven (Kafka), competing consumers (notification workers), idempotent consumer (duplicate events), dead-letter queue, and claim-check (large attachments via storage).

Platform-specific — Dynamic RBAC (roles, permissions, and modules as data), approval / governance workflow, and signed-URL media access.

3. Everything Needed — Stack & Tooling

Frontend — Next.js (App Router) with JavaScript, Tailwind CSS, React Query, and NextAuth / SSO.

Backend — Java with Spring Boot, Spring Cloud Gateway, Spring Security (method-level), Spring Data JPA, Resilience4j, Spring Cloud Config, and Eureka or Consul.

Data — PostgreSQL (per service), Redis, Elasticsearch, and object storage (S3 / MinIO) with a CDN.

Messaging — Apache Kafka with a schema registry.

Reporting — JasperReports (PDF / XLSX / DOCX).

Auth & security — JWT, OAuth2 / OIDC, SSO / SAML, 2FA / TOTP, a secrets manager (Vault), TLS, and audit logging.

Notifications — Email (SES / SendGrid), WhatsApp Business API, and an SMS gateway.

Media — FFmpeg for HLS transcoding, signed URLs, and a CDN.

DevOps — Docker, Kubernetes, Helm, a CI/CD pipeline, and Terraform for infrastructure.

Observability — Prometheus and Grafana, centralised logging (ELK or Loki), distributed tracing (OpenTelemetry / Jaeger), and alerting.

Testing — Unit and integration tests, consumer-driven contract tests, and load testing.